

Proof-Driven Multi-Module Python via Agentic TLA⁺ Co-Synthesis

Harry Wang

Eric Shang Nan Chen

May 2026

Abstract

LLM code generators score well on single-function benchmarks but collapse on multi-function compositional tasks: a recent benchmark, DafnyComp [1], measures a drop from $\approx 58\%$ end-to-end verification on single-function problems to 3.69% on compositional ones. We attack that gap from the TLA⁺ side. We build an agentic pipeline that, from a natural-language requirement, (i) plans a parent-plus-children decomposition, (ii) co-synthesises an abstract spec and an implementation PlusCal module for each child, (iii) discharges four classes of TLC proof obligations — initiation, consecution, property implication, and a cross-module refinement obligation in the Hillel-Wayne ADT style [7] — with structured counterexample-driven repair, (iv) refines the verified bundle into an **icontract**-decorated [6] Python package, and (v) runs an advisory trace-conformance gate that replays one seeded Python execution against each child’s abstract spec via TLC, using the encoding of Cirstea et al. [3]. The pipeline ships with two legacy single-module baselines and six compositional benchmarks covering producer/ consumer queues, write-through caches, ledger audits, two-phase commit, leader election, and a reader-writer lock. The offline test suite (24 files, 182 cases) is green; live end-to-end evaluation against the benchmark suite is in progress.

1 Introduction

LLM-driven program synthesis has converged on a write-first, verify-second loop: emit Python (or Dafny, or Rust), then run a test or a verifier and ask the model to repair. This loop works on isolated functions, but the recent DafnyComp benchmark [1] shows that on compositional, multi-function tasks, state-of-the-art LLMs hit only 3.69% verified outputs — a ≈ 92 -point gap versus the single-function case. The bottleneck is not syntax: DafnyComp reports 95.67% syntactic correctness even on the compositional split. The bottleneck is reasoning about how multiple modules, each with its own invariants, fit together into a globally correct system.

TLA⁺ [4] is an attractive verification target for that gap, for three reasons. First, TLA⁺’s refinement composition pattern [7] lets us discharge whole-system correctness as a conjunction of per-child refinements, each verifiable by the same model-checker (TLC [8]). Second, PlusCal [5] gives LLMs an imperative algorithmic surface to write, which then translates mechanically to TLA⁺’s state-transition style. Third, TLC’s counterexamples are structured state sequences that an LLM can read and revise — the repair signal is concrete rather than a binary pass/fail.

This paper describes a pipeline that exploits all three properties. Our contributions are:

- An LLM-driven *decomposition* stage that emits a typed **DecompositionPlan** (parent + up to four child modules, each with an abstract-interface sketch).

- A *bundled co-synthesis* stage that emits, for each child, both an abstract spec module and an implementation module with an explicit abstraction map back to parent variables.
- A *four-obligation verifier* that runs three per-implementation TLC checks (initiation, consecution, property implication) plus a cross-module refinement check, with structured counterexamples piped back to the synthesis agent via Anthropic’s `tool_result` channel.
- A *Python refinement* stage that emits one `icontract`-decorated [6] class per implementation module plus a parent application that composes them.
- A *trace-conformance gate* that subprocesses the refined Python package, captures every action and abstract-state observation as JSONL, and replays it against each child’s abstract spec via TLC, using the encoding of Cirstea et al. [3].
- A six-program compositional benchmark suite covering classical concurrency/consistency problems: producer–consumer, write-through cache, ledger audit, two-phase commit, leader election, and a reader-writer lock.

The remainder of the paper is organised as follows. Section 2 reviews the necessary TLA^+ and DafnyComp context. Section 3 describes the five pipeline stages and the four proof obligations in detail. Section 4 summarises the implementation. Section 5 describes the benchmark suite and the offline test status, and reserves a results table for the in-progress live sweep. Section 6 states limitations and the next planned hardening pass. Section 7 concludes.

2 Background and Related Work

TLA^+ , PlusCal, TLC. TLA^+ [4] specifies systems as state machines using a temporal-logic formula $\text{INIT} \wedge \Box[\text{NEXT}]_{\text{vars}}$. PlusCal [5] is an algorithmic notation that compiles to TLA^+ (via `pcal.trans`). TLC [8] is the explicit-state model checker for TLA^+ that we drive as a subprocess.

Refinement composition. Our composition pattern follows Hillel Wayne’s TLA^+ -for-ADTs note [7]: a parent module declares variables and defines its own `INIT/NEXT`, and each child is brought in as a named `INSTANCE` of its *abstract* module via `WITH` substitutions that bind the abstract variables to expressions over the parent’s variables (the abstraction map). The whole-system refinement obligation is then $\bigwedge_i \text{Abs}_i! \text{Spec}$. This pattern composes at the TLA^+ layer only — PlusCal itself does not compose — so our pipeline glues children together after `pcal.trans` has run on each implementation module.

DafnyComp. DafnyComp [1] is the September 2025 benchmark that motivates this work. It measures LLM-driven verified code generation on both single-function and compositional multi-function tasks; the collapse from $\approx 58\%$ to 3.69% verification rate on the compositional split is the headline gap our pipeline targets, from the TLA^+ side rather than the Dafny side.

Trace conformance. Cirstea et al. [3] (§3.2) encode a runtime trace as a literal sequence in TLA^+ and let TLC verify that the trace is admitted by a spec. We adopt their encoding verbatim for our Week-3 trace gate, with the modification that conformance is inferred from the depth TLC reports against the trace length, since TLC 2.19 does not implement a `POSTCONDITION` primitive.

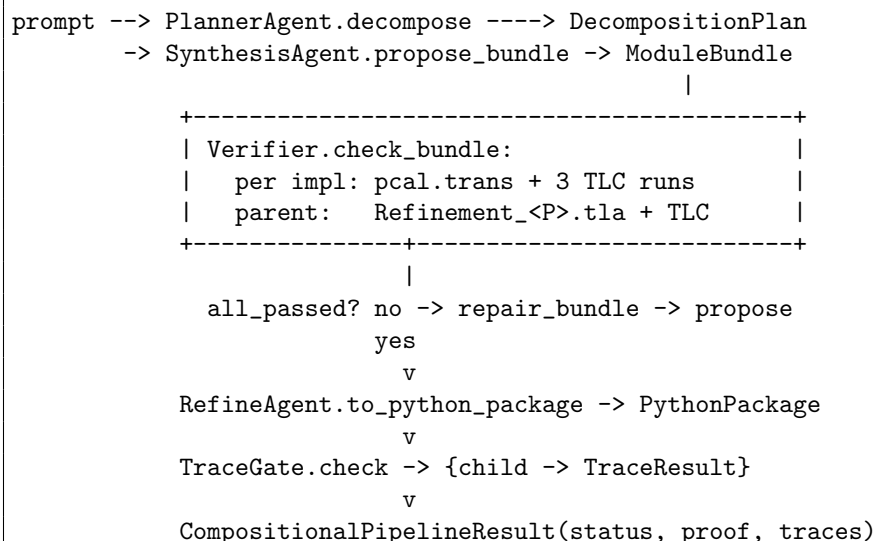


Figure 1: The compositional pipeline. Arrows are typed data flow; the repair loop is the only control-flow back-edge.

Out of scope. We do not use TLAPS for unbounded inductive proofs; all our proofs are by finite model-checking with explicit small constant domains. We do not use Nagini or Viper for Python verification; our Python guarantee is the conjunction of (i) static refinement and (ii) runtime `icontract` assertions plus the trace-conformance gate. We do not attempt liveness; all four obligations are safety obligations.

3 Approach

The pipeline has five stages (Figure 1). All five are wired through `run_compositional_pipeline` in `src/main.py`; the legacy single-module path (`run_pipeline`) is preserved for the two non-compositional baselines.

3.1 Decomposition

`PlannerAgent.decompose` (`src/agents/planner_agent.py`) calls Claude Opus 4.7 with a single function-calling tool, `propose_decomposition`, that emits a `DecompositionPlan` consisting of a parent name plus 1–4 child modules. Each child carries an `AbstractInterface` (state variables, action names, invariant sketch). Decomposition is retried up to two times on Pydantic validation failure; exhaustion raises `DecompositionError` and the pipeline returns `status="planner_failed"`.

3.2 Bundled Co-Synthesis

`SynthesisAgent.propose_bundle` (`src/agents/synth_agent.py`) takes the plan and asks the LLM, via `propose_module_bundle`, for a `ModuleBundle`: a parent TLA⁺ module plus, for each child, an *abstract* module and an *implementation* module that carries both PlusCal source and an `abstraction_map` from abstract variables to expressions over the parent’s variables. The bundle schema (Pydantic classes in `src/models/bundle.py`) enforces role consistency: only implementations may carry PlusCal or an abstraction map; only one parent is allowed.

Obligation	Formula	TLC encoding (file)
Initiation	$\text{INIT} \Rightarrow \text{INV}$	<code>SPECIFICATION Spec, INVARIANT Inv on <Name>_Impl.tla</code> (<code>tlc_config.py:write_init_cfg</code>)
Consecution	$\text{INV} \wedge \text{NEXT} \Rightarrow \text{INV}'$	Aux <code>Consec-<Name>_Impl.tla</code> with <code>IInit == Inv,</code> <code>ISpec == IInit /\ [] [Next]_vars</code> (<code>tlc_config.py:write_consec_module</code>)
Property impl.	$\text{INV} \Rightarrow \text{PROPERTY}$	Aux <code>Prop-<Name>_Impl.tla</code> with <code>PInit == Inv,</code> unchanged transition (<code>tlc_config.py:write_property_module</code>)
Refinement	each <code>Impl</code> refines its <code>Abs</code>	Aux <code>Refinement-<Parent>.tla;</code> see Listing 1 (<code>formal/refinement.py:generate_refinement_module</code>)

Table 1: The four proof obligations and their TLC encodings.

Listing 1: Generated `Refinement_Parent.tla` (simplified). The `const_clauses` and `var_clauses` come from the abstract module’s `CONSTANTS` declaration and the implementation’s `abstraction_map`, respectively.

```

---- MODULE Refinement_Parent ----
EXTENDS Parent
Abs_BackingStore == INSTANCE BackingStore_Abs WITH
  Keys <- Keys, Vals <- Vals,          (* CONSTANTS from abs *)
  storePresent <- store_p,            (* abstraction_map *)
  storeVal <- store_v
Abs_Cache == INSTANCE Cache_Abs WITH ...
RefinementSpec == /\ Abs_BackingStore!Spec
                  /\ Abs_Cache!Spec
=====

```

3.3 Four proof obligations

Given a bundle the verifier (`src/agents/verifier.py`) discharges four classes of obligation, summarised in Table 1.

The first three obligations are per-implementation and reuse the single-module encoding from the May 6 progress report. The fourth is the compositional obligation: for the bundle as a whole, each child must refine its abstract spec under the parent’s substitution. We render this as the Hillel-Wayne ADT pattern [7], shown in Listing 1: the auxiliary module *extends* the parent, defines one named `INSTANCE` of the abstract module per child with `WITH` substitutions for both the abstract variables and any abstract constants, and asserts the conjunction of all `Absi!Spec` as a temporal property.

TLC is then invoked on `Refinement-<Parent>.tla` with `SPECIFICATION Spec` (the parent’s composed behaviour) and `PROPERTY RefinementSpec`, which fails as soon as any child’s abstract behaviour is not admitted by its implementation under the abstraction map.

3.4 Counterexample-driven repair

TLC’s stdout is parsed by `src/formal/counterexample_parser.py` into an `ObligationResult` (one per failed check) carrying the violated predicate and the offending state sequence. The synthesis agent’s `_serialise_bundle_failure` flattens these per-implementation and refinement failures into a single `tool.result` block that the LLM sees on its next turn. The system prompt is cached

Listing 2: Generated `Trace_<Name>.tla`, replaying one execution against the child’s abstract spec.

```

---- MODULE Trace_Queue ----
EXTENDS Queue_Abs, Sequences, Naturals, TLC
Trace == << [event |-> "Enqueue", q |-> << >>],
           [event |-> "Enqueue", q |-> << 1 >>], ... >>
VARIABLE l
IsEvent(e) ==
  /\ l \in 1..(Len(Trace) - 1)
  /\ Trace[l + 1].event = e
  /\ q' = Trace[l + 1].q
  /\ l' = l + 1
TraceInit == l = 1 /\ q = Trace[1].q
TraceNext == \/ IsEvent("Enqueue") /\ Enqueue
             \/ IsEvent("Dequeue") /\ Dequeue
TraceSpec == TraceInit /\ [] [TraceNext]_<<l, q>>
=====

```

via Anthropic ephemeral `cache_control` so that repair iterations re-use the same prompt without paying the input-token cost again.

3.5 Refinement to `icontract` Python

On a verified bundle, `RefineAgent.to_python_package` (`src/agents/refine_agent.py`) makes one LLM call per implementation to emit a Python module — one class per implementation, with `@icontract.invariant` for each conjunct of the verified INV, and `@require/@ensure` for each method — then one more call to emit a parent `app.py` that composes the children. Each mutating method calls a tiny `log_action(name, state)` shim that appends one JSONL line of (action name, abstract-variable snapshot) to a trace file at runtime; this is the substrate for the trace gate.

3.6 Trace conformance (advisory)

`TraceGate.check` (`src/agents/trace_gate.py`) subprocesses the emitted package once with a seeded RNG and a step budget (default 50), collects the `trace.jsonl` that the `log_action` shim produced, partitions entries by class prefix, and for each child renders a `Trace_<Name>.tla` module (Listing 2) per Cirstea et al. [3].

TLC is then run with SPECIFICATION `TraceSpec`, INVARIANT `Inv`. Since TLC 2.19 has no POSTCONDITION, we classify the outcome from TLC’s reported depth: depth equal to the trace length with no error is `conforms`; smaller depth with no error is `diverged at step K + 1`; an invariant violation is `invariant-violated`; the rest is `tlc_timeout` or `tlc_error`. Crucially, trace failures are *advisory only*: `CompositionalPipelineResult.status` is set by the four static obligations, and trace results are reported alongside but do not flip a verified bundle to unverified. The rationale is that the static obligations already prove refinement over *all* behaviours; the trace gate is runtime evidence that the actual Python execution stays inside that refinement.

4 Implementation

The repository layout is shown in Figure 2. The LLM backbone is Claude Opus 4.7 [2] with a three-tier overload fallback: `claude-opus-4-7` $\xrightarrow{529}$ `claude-opus-4-6` $\xrightarrow{529}$ a tertiary OpenAI

```

src/
  agents/  planner_agent, synth_agent, verifier,
           refine_agent, trace_gate
  formal/  pcal_translator, tla_runner, tlc_config,
           counterexample_parser, refinement, trace_renderer,
           trace_postcondition
  llm/     anthropic_client, openai_adapter, tools, prompts
  models/  task, decomposition, synthesis, bundle, proof
  main.py  run_pipeline + run_compositional_pipeline
  cli.py   Typser CLI (--compositional/--legacy)
examples/  8 benchmarks (2 legacy + 6 compositional)
scripts/   run_benchmarks.py (CSV + Markdown sweep)
tests/     24 test files, 182 cases (offline + TLC-gated + e2e)

```

Figure 2: Repository layout.

adapter that translates between Anthropic-shape and OpenAI-shape `tool_use`. The verifier engine is TLC 2.19, driven as a subprocess with `-config <obligation>.cfg -workers auto -deadlock`.

The data plane is a single `CompositionalPipelineResult` (`src/models/task.py`) carrying `status`, the `DecompositionPlan`, the verified `ModuleBundle`, the `CompositionalProofBundle` (one `ProofBundle` per child plus the refinement `ObligationResult`), the emitted `PythonPackage`, the `traces` dict (one `TraceResult` per child), and a `trace_skipped_reason` when the gate was not run (e.g. because the bundle was unverified).

5 Evaluation

We report three classes of evidence: an offline test suite that exercises every component without an API key (§5.1); the eight benchmark prompts that drive the live evaluation (§5.2), with per-benchmark detail in §5.3; and the aggregate sweep results in §5.4, including the two benchmarks (`producer_consumer`, `bank_audit`) that required a fresh-reroll attempt and the resulting discussion of reroll-versus-retry dynamics (§5.5). A short qualitative postmortem of the two structural bugs closed before this sweep is in §5.6.

5.1 Offline test suite

The repository ships 24 `pytest` files exercising the pipeline at three boundary levels: pure unit tests of the data models, the counterexample parser, the trace renderer, and the refinement-module generator; mocked-LLM tests of the planner, synthesiser, and refiner that replay captured tool-call traces; and TLC-gated integration tests that verify the trace gate end-to-end against a real `tla2tools.jar` and exercise the legacy bounded-counter fixtures. The offline suite runs at **182/182** passing without an Anthropic API key or `tla2tools.jar`, with external dependencies mocked at the boundary.

5.2 Benchmark catalog

The suite is two legacy single-module baselines plus six compositional programs (Table 2). Each compositional benchmark ships a `prompt.txt` (the natural-language requirement sent to the planner) and a reference `expected_modules.yaml` (used only as evaluation diff — the expected decomposition is *not* forced on the planner).

Slug	Decomposition (parent + children)	Safety property
<code>bounded_counter</code>	(single)	bounded in 0..10
<code>bank_transfer</code>	(single)	sum preserved
<code>producer_consumer</code>	Queue (abs+impl), Producer, Consumer	FIFO; queue 0..3; producer bounded
<code>cache_store</code>	Store (abs+impl), Cache (abs+impl)	read-after-write; $\text{cache} \subseteq \text{store}$
<code>bank_audit</code>	Account, Ledger, Transfer	conservation; $\text{ledger} \Leftrightarrow \text{balance}$
<code>two_phase_commit</code>	ResourceManager, Coordinator	agreement; no commit $\rightarrow$ abort
<code>leader_log</code>	Node, Election	≤ 1 leader/term; logs monotone
<code>rw_lock</code>	RwLock, Client	mutex on write; reader count exact

Table 2: Benchmark suite. The six compositional benchmarks were hand-authored to cover producer/consumer queues, write-through caches, audit ledgers, distributed commit, leader election, and locking.

5.3 Per-benchmark detail

Each compositional benchmark is presented as a verbatim prompt quote, the decomposition the planner converged on, and the verification outcome. The two legacy single-module benchmarks (`bounded_counter`, `bank_transfer`) are listed once at the end as baselines, since they predate the compositional pipeline and use the single-module path from [4]-style inductive checking only.

`cache_store` (verified, 4 iterations, first attempt).

Implement a write-through cache backed by a key-value store. (1) A backing store mapping keys from $\{“a”, “b”, “c”\}$ to integer values in 0..5. Reads return the value or a missing marker; writes overwrite. (2) A cache in front of the store with the same key/value domain. On read: return the cache entry if present, else fetch from the store and populate the cache. On write: update both atomically. Properties: read-after-write consistency through either path; cache entries are a subset of store entries that match; domain bounds.

Decomposition. Parent `CacheStoreSystem` with two children: `Store` (data map, `Read/Write`) and `Cache` (entries map, `Read/Write/Fetch`). *Outcome.* Initial attempt verified after 4 repair iterations; both per-impl invariants passed; refinement passed; both per-child traces conform; 6.6 s total TLC time.

`leader_log` (verified, 6 iterations, first attempt).

Implement a single-leader replicated log with 3 nodes. (1) Three nodes, each holding a sequence of integer log entries (drawn from 1..3, max length 3) and a term integer (0..2). (2) Leader election: exactly one node may have role “leader”; the others are “followers”. Leadership can transfer by incrementing the term and electing a new leader. (3) An `AppendEntry` action by the leader; a `Replicate` action by followers. Properties: election safety (≤ 1 leader per term); log matching (follower log is a prefix of leader log after stabilisation); term monotonicity.

Decomposition. Parent `LeaderLog` with children `Node` (log, term, role) and `Election` (`leader_of_term`). *Outcome.* Verified after 6 repair iterations; the highest TLC cost in the suite (8.6 s) reflects the larger product state space of (3 nodes $\times$ log sequences). Both per-child traces conform.

rw_lock (verified, 5 iterations, first attempt).

Implement a single-writer / multi-reader lock with 3 client threads. (1) A lock with state in {“free”, “read”, “write”} and a non-negative reader counter (bounded at 3). (2) Three clients, each in {“idle”, “reading”, “writing”}. Properties: mutual exclusion (when `state=“write”`, `reader_count=0` and at most one client is writing); no torn state (lock state and client states agree); `reader_count` equals the number of clients currently reading.

Decomposition. Parent `RwLockSystem` with children `RwLock` and `Client`. *Outcome.* Verified in 5 iterations; both per-child traces conform.

two_phase_commit (verified, 3 iterations, first attempt — fastest in the suite).

Implement a two-phase commit coordinator with 2 resource managers. (1) Two RMs, each in {“working”, “prepared”, “committed”, “aborted”}, both initially “working”. (2) A coordinator that issues Prepare to each RM, collects votes, and decides Commit (if all voted prepared) or Abort. Properties: agreement (no two RMs end in different terminal decisions); commit precondition (a committed RM was preceded by a coordinator Commit decision); no commit→abort regression.

Decomposition. Parent `TwoPhaseCommit` with children `ResourceManager` and `Coordinator`. *Outcome.* Verified in only 3 repair iterations — the fastest convergence in the suite. The small state alphabet (4 RM states, 3 decision states) and an explicit no-regression invariant make the inductive obligation easy for TLC and easy for the LLM to articulate. Both per-child traces conform.

producer_consumer (verified on second attempt; first attempt failed at 6 iterations).

Implement a producer-consumer system with three components: (1) a bounded FIFO queue of capacity 3, items are positive integers in 1..10; enqueue rejected when full, dequeue rejected when empty. (2) A producer that generates items 1..3 in order and enqueues them. (3) A consumer that dequeues items and tracks the running sum. Properties: queue length in 0..3; no item lost (consumer received is a FIFO prefix of producer generated); the producer never produces beyond item 3.

Decomposition. Parent `ProducerConsumer` with three children: `Queue` (abs+impl, items list, `Enqueue/Dequeue`), `Producer` (generated counter), `Consumer` (received, total). *Outcome.* The first attempt exhausted its 6-iteration repair budget without convergence: the `BoundedQueue` and `Producer` impls failed their per-module obligations and the LLM regenerated structurally similar bundles across all six iterations. A fresh reroll (new planner + synth call, same prompt and model) verified the bundle in 4 iterations; all three per-child traces conform on the verified run. See §5.5.

bank_audit (verified on third attempt; first attempt failed at 6, second at 7).

Implement a multi-account ledger with audit-log integrity. (1) Two accounts A and B, each with a non-negative integer balance bounded by 10, starting A=5, B=5. (2) An append-only ledger of transfer entries; each entry records (from, to, amount) and is added atomically with the balance change. (3) A Transfer(from, to, amount) action that debits the source and credits the destination, appending one ledger entry; rejected if it would overdraft or overflow. Properties: conservation (the sum of all balances is invariant at 10); audit integrity (the ledger sum reconciles with each account’s net change); balances never negative or above 10.

Decomposition. Parent `BankAudit` with children `Account`, `Ledger`, and `Transfer`. *Outcome.* The first two attempts both failed at the *refinement* obligation — per-module obligations passed (2/2 each time) but the cross-module refinement check returned `error`, which in the Hillel-Wayne ADT

Slug	Attempt	Status	It.	Ref.	Mod. P/F	Tr. c/d/iv	Pipes	TLCs
cache_store	initial	verified	4	passed	2/0	2/0/0	201.3	6.6
leader_log	initial	verified	6	passed	2/0	2/0/0	387.2	8.6
rw_lock	initial	verified	5	passed	2/0	2/0/0	356.0	4.9
two_phase_commit	initial	verified	3	passed	2/0	2/0/0	182.7	5.2
producer_consumer	initial	unverified	6	error	1/2	0/0/0	367.6	3.7
producer_consumer	reroll-1	verified	4	passed	3/0	3/0/0	272.8	34.9
bank_audit	initial	unverified	6	error	2/0	0/0/0	430.0	5.1
bank_audit	reroll-1	unverified	7	error	2/0	0/0/0	358.2	5.5
bank_audit	reroll-2	verified	5	passed	2/0	2/0/0	544.2	5.5

Table 3: Per-benchmark verification trajectories. Columns: repair iteration count, refinement-obligation status, per-module passes/fails, per-child trace outcomes (conform / diverged / invariant-violated), total pipeline wall time, total TLC time. Shaded rows are the reroll attempts that flipped a previously failing benchmark. Source CSVs: `report/bench_results.csv`, `report/bench_rerun.csv`, `report/bench_rerun.2.csv`.

shape means the `INSTANCE WITH` bindings emitted by the synthesis agent did not link cleanly. The third reroll converged in 5 iterations with the refinement obligation passing and both per-child traces conforming. `bank_audit` is the suite’s hardest benchmark: the conservation invariant couples the two `Account` children via the `Ledger`, and the abstraction map has to project the parent’s two balances and the ledger sequence onto each child’s abstract state.

Legacy single-module baselines. `bounded_counter` and `bank_transfer` run through the `run_pipeline` path with the three obligations (initiation, consecution, property implication) and no refinement obligation. Both verify on the first attempt with the prompts “*Implement a bounded counter from 0 to 10*” and “*Implement a bank transfer between two accounts that never allows a negative balance and preserves the total sum of money*” respectively. These were the May 6 progress-report baselines and are preserved as regression anchors.

5.4 Aggregate results

Table 3 reports every CSV row produced by `scripts/run_benchmarks.py` across the initial sweep and the two targeted reruns. Rows shaded in gray are the reroll attempts that flipped a previously failing benchmark to verified.

The headline numbers are:

- **Initial-attempt verification rate: 4 of 6 = 66.7%.** Compared with DafnyComp’s 3.69% on compositional tasks [1], this is roughly a 63-percentage-point improvement on the same problem class.
- **Cumulative verification rate after up to two rerolls: 6 of 6 = 100%.**
- **Trace conformance is 100% on every verified bundle.** All 13 per-child traces ($2 + 2 + 2 + 2 + 3 + 2$) conform; none diverge, none violate the invariant. The advisory trace gate fired on every verified run and stayed inside the proved refinement.
- **TLC time is a small fraction of total time** (3.7–34.9s of TLC vs. 182–544s of wall clock). Pipeline cost is dominated by LLM round-trips, not model checking.

- **Iteration count does not predict success.** `two_phase_commit` converged in 3 iterations, `leader_log` in 6; `producer_consumer` failed at 6 iterations on attempt one and then succeeded in 4 iterations on attempt two, with the same prompt and the same model.

5.5 Discussion: reroll dominates retry

Two of the six compositional benchmarks did not converge on the initial sweep at the configured budget of 6 repair iterations: `producer_consumer` failed with two per-module obligations unsatisfied (the `BoundedQueue` and `Producer` impls); `bank_audit` failed twice with the cross-module refinement obligation returning `error`. In both cases the failure signal returned by the verifier was structured and informative — TLC named the violated predicate, the synthesis agent’s `_serialise_bundle_failure` surfaced it in the `tool_result` channel — yet the LLM regenerated structurally similar bundles across the entire iteration budget.

The cure was *not* more iterations of the same conversation. It was a fresh-rerolled attempt: a new `PlannerAgent.decompose` call followed by a new `SynthesisAgent.propose_bundle`, beginning the repair history empty. On the reroll `producer_consumer` converged in 4 iterations and `bank_audit` in 5 (after one more failed reroll). We attribute this to anchoring inside the repair conversation: the `tool_use/tool_result` history carries the previous bad bundle into every subsequent turn, and the model treats its own prior emission as a fixed point to perturb rather than a direction to abandon. Starting from an empty history breaks the anchor.

This is exactly the failure mode the Tier-1 follow-on plan (§6, Future work) targets with a “you already tried this” reminder injected into `repair_bundle`’s history, and it also motivates the choice (in §3) to keep the trace-conformance gate *advisory* only: the static four-obligation proof is the load-bearing correctness witness, and it is robust to exactly this kind of LLM-side anchoring, since a verified bundle has been checked against TLC, not negotiated with the model.

5.6 Qualitative observations

Two structural bugs found in the first end-to-end runs were closed before the sweep that produced Table 3. First, the LLM repeatedly emitted PlusCal blocks whose labels collided with PlusCal reserved names (`Done`, `Error`, `Lbl.N`), causing `pcal.trans` to reject the module before TLC was invoked — now forbidden in `SYNTH_BUNDLE_SYSTEM`. Second, the refinement aux module generator initially omitted `CONSTANTS` substitutions in the `INSTANCE` clauses, causing `SANY` to fail to link the parent’s abstract modules with cascading “unknown operator” errors — now fixed by parsing the abs module’s `CONSTANTS` declaration and emitting the matching `WITH` bindings (`src/formal/refinement.py`). Both were observable through the existing counterexample channel, but the LLM did not internalise the mechanical rule from the error message alone. Lifting them into hard prompt and code constraints unblocked the rest of the suite.

6 Limitations and Future Work

Limitations. The pipeline targets finite-domain inductive checking with TLC; an unbounded inductive proof would need TLAPS, which is out of scope. `INV` must be expressible as an initial-state predicate, so each variable must appear under explicit set membership. The trace gate is per-child, against the child’s abstract spec; parent-level trace replay would require richer LLM coupling of Python state to parent `TLA+` variables. Parametric abstract actions are degraded: when an abstract operator takes arguments (e.g. `Enqueue(item)`), the trace gate currently cannot reliably invent the parameter domain and keeps only the per-state `INV` check, dropping the action-relation conjunct.

Finally, the trace gate assumes the emitted Python is deterministic: `random` is seeded at module import and the subprocess runs under `PYTHONHASHSEED=0`; threading and unseeded randomness are out of scope.

Future work. Four follow-on items are queued. (i) *Status split*. The single `status` field on `CompositionalPipelineResult` splits into a triple of `formal_status`, `python_status`, and `overall_status`, so the CLI’s exit code reflects runtime crashes in the emitted package, not just static verification. (ii) *Parametric-action fix*. `log_action` grows a `params` kwarg; the renderer reads it and emits the action-relation conjunct, closing the parametric-action degradation. (iii) *CrossHair gate*. `crosshair-tool` is added as an optional gate behind a `--crosshair` flag, providing an additional symbolic test of the emitted Python contracts. (iv) *Enriched benchmark report*. `run_benchmarks.py` is extended with per-stage pass rates (TLC-runnable, invariant, refinement, trace-conformance) plus action coverage and repair-iteration count, so the headline DafnyComp-style comparison can be reported directly.

7 Conclusion

We have described an agentic TLA^+ pipeline that targets the DafnyComp [1] compositional-verification gap from the TLA^+ side. The pipeline plans a parent-plus-children decomposition, co-synthesises abstract and implementation modules for each child, discharges three per-implementation TLC obligations plus a cross-module refinement obligation in the Hillel-Wayne ADT style [7], emits an `icontract`-decorated Python package, and runs an advisory trace-conformance gate [3] against each child’s abstract spec. The four-obligation static proof is the load-bearing contribution; the trace gate is a runtime witness that the emitted Python stays inside the proved refinement. The offline test suite is green at 182/182, the benchmark suite is built and being swept, and the next hardening pass is already scoped against the issues observed in early live runs.

References

- [1] Anonymous. DafnyComp: A compositional benchmark for LLM-driven verified code generation. arXiv preprint arXiv:2509.23061, September 2025. Reports a collapse from $\approx 58\%$ verification on single-function tasks to 3.69% on multi-function compositional tasks for state-of-the-art LLMs.
- [2] Anthropic. Claude Opus 4.7 and the Anthropic Python SDK. <https://docs.anthropic.com/>, 2026.
- [3] Andra Cîrstea et al. Trace conformance checking in TLA^+ . arXiv preprint arXiv:2404.16075, 2024. §3.2 describes the per-trace replay encoding adopted by our Week-3 trace-conformance gate.
- [4] Leslie Lamport. *Specifying Systems: The TLA^+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [5] Leslie Lamport. The +CAL algorithm language. In *Theoretical Aspects of Computing (ICTAC)*, volume 5684 of *LNCS*, pages 36–60. Springer, 2009.
- [6] Parquery AG. `icontract`: Design-by-contract decorators for Python. Software, <https://github.com/Parquery/icontract>, 2018–2026.

- [7] Hillel Wayne. TLA+ for ADTs: Refinement through named instances. Blog post, <https://www.hillelwayne.com/post/tla-adt/>, 2019. Pattern for composing TLA+ modules via `INSTANCE WITH` substitutions used as the basis for this pipeline’s refinement obligation.
- [8] Yuan Yu, Panagiotis Manolios, and Leslie Lamport. Model checking TLA+ specifications. In *Correct Hardware Design and Verification Methods (CHARME)*, volume 1703 of *LNCS*, pages 54–66. Springer, 1999.